IBM CASE STUDY

Banking Fraud Pipeline with Outlier Capping & PR Diagnostics

Name	Koripala Vishal
Roll No	A23126552214
IBM ID	IBMP4072
Department	CSM
College Name	Anil Neerukonda Institute Of Technology & Sciences
Module Name	Artificial Intelligence & Machine Learning
Module Number	22
UG Level	U3

Table of Contents

S.No	Topic	Page No.
1	Case Study Opted	3
2	Overview	3
3	Business Problem	3
4	Dataset	4
5	Evaluation Metrics Defined (KPIs)	4
6	Pipeline & System Design	5
7	Key Insights Surfaced	8
8	Business Impact and Recommendations	11
9	Challenges and Solutions	11
10	Outcome of Case Study	11
11	Conclusion	12

1. Case Study Opted

Case study title: Banking Fraud Pipeline with Outlier Capping & PR Diagnostics

Credit card fraud datasets carry two properties that break naive machine learning workflows: extreme numerical outliers in transaction amounts, and a fraud rate low enough (well under 1%) that standard accuracy becomes meaningless. In this project, a leakage-safe Scikit-learn pipeline is built, evaluated with an honest held-out test split, made explainable with SHAP, and deployed as a real-time REST API, using the Credit Card Fraud Detection dataset.

2. Overview

Fraud review teams need more than a model that scores well in a notebook: they need a system whose reported performance can be trusted on transactions it has never seen, and whose flags come with a reason a human reviewer can act on quickly. Two failure modes are common in fraud modeling projects: evaluating on accuracy when the positive class is a fraction of a percent of the data, which rewards a model that flags nothing at all, and fitting preprocessing steps such as scaling or outlier handling on the full dataset before splitting, which quietly leaks test-set information into training and inflates reported performance in a way that will not survive contact with new data.

This case study documents the design, build, and deployment of a fraud detection pipeline that addresses both failure modes directly, using the ULB Credit Card Fraud Detection dataset, a real anonymized transactional dataset covering 284,807 European card transactions over two days in September 2013.

- **Tools used:** Python, Scikit-learn, SHAP, FastAPI, Docker
- **Dataset:** Credit Card Fraud Detection Dataset (ULB Machine Learning Group, via Kaggle)
- **Deliverable:** a tested, containerized REST API that scores transactions in real time and explains every prediction

3. Business Problem

Fraud detection systems built without care around evaluation and data hygiene create three recurring problems:

- **Misleading evaluation:** with fraud at 0.173% of transactions, a model that flags nothing at all still scores 99.8% accuracy — high accuracy and zero usefulness look identical on that one metric.
- **Leakage-prone preprocessing:** imputing, scaling, or capping outliers using statistics computed from the full dataset (including what will become the test set) inflates validation scores in a way that does not transfer to production traffic.
- **Opaque decisions:** a flagged transaction with no explanation is slow for a fraud analyst to act on, and difficult for a regulator or auditor to review after the fact.

Goal: build a pipeline whose reported performance survives an honest, completely untouched holdout test, and whose every decision can be explained to a human reviewer in the transaction's own original units, not in scaled, unreadable numbers.

4. Dataset

The ULB Credit Card Fraud Detection dataset (Kaggle: mlg-ulb/creditcardfraud; Dal Pozzolo et al.) contains transactions made by European cardholders. To preserve confidentiality, 28 of the 30 input features (V1–V28) are the output of a PCA transformation applied by the original researchers; only Time (seconds elapsed since the first transaction) and Amount (transaction value) remain in their original, human-readable units.

Total transactions	284,807
Confirmed fraud	492 (0.173%)

Input features	30 — Time, V1–V28 (PCA-anonymized), Amount
Time span covered	≈ 48 hours
Missing values	0

5. Evaluation Metrics Defined (KPIs)

Where a business dashboard tracks revenue or retention, this pipeline is judged against metrics chosen specifically so the 0.173% fraud rate cannot flatter a lazy model:

Metric	Definition	Why it's tracked
PR-AUC (Average Precision)	Area under the precision-recall curve	Primary metric — not inflated by the large majority class the way accuracy or even ROC-AUC can be
ROC-AUC	Area under the ROC curve	Tracked specifically to expose the gap against PR-AUC (see Key Insights)
Precision @ threshold	Share of flagged transactions that are real fraud	Determines wasted review effort per flag
Recall @ threshold	Share of real fraud caught	Determines how much fraud is missed entirely
Scoring latency	Time to score + explain one transaction	Operational KPI for real-time usability

Achieved values, measured once on a 56,962-transaction held-out test set (98 confirmed fraud) that was never used in model selection, threshold tuning, or fitting:

Metric	Achieved value (held-out test set)
PR-AUC (Average Precision)	0.708
ROC-AUC	0.971
Precision @ deployed threshold	83.2%
Recall @ deployed threshold	80.6%
F1 @ deployed threshold	0.819
Scoring latency (incl. explanation)	~15 ms

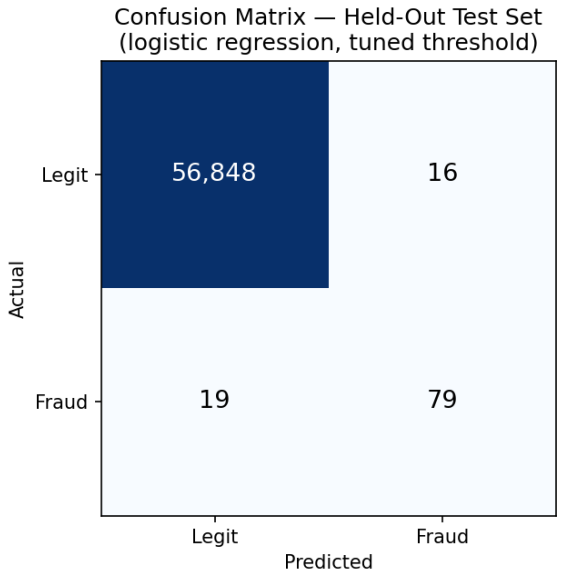


Figure 1 — Confusion matrix on the held-out test set (logistic regression, tuned threshold). 79 of 98 fraud cases caught; 16 false alarms out of 56,864 legitimate transactions.

6. Pipeline & System Design

In place of a Power BI / Tableau dashboard, this case study's front end is a documented, tested REST API sitting on top of a leakage-safe Scikit-learn pipeline. The design follows four layers:

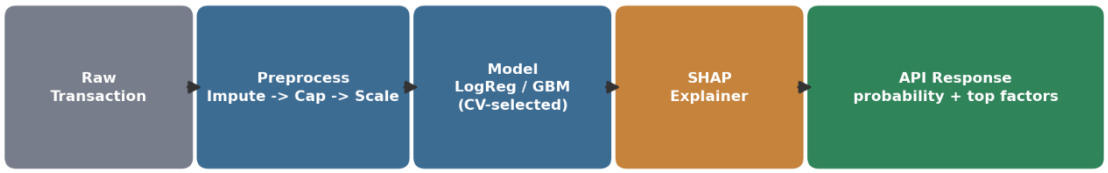


Figure 2 — End-to-end request flow, from raw transaction to explained prediction.

- **1. Preprocessing (ColumnTransformer):** median imputation everywhere as a defensive step; IQR-based outlier capping applied only to Amount, not to V1–V28 (fraud signal in this dataset disproportionately lives in the tails of those anonymized PCA components, so capping them would remove the exact signal the model needs); RobustScaler (median/IQR-based) throughout, since it stays stable in the presence of the outliers this project is explicitly about.
- **2. Model selection:** two candidate pipelines — Logistic Regression with class_weight="balanced", and HistGradientBoostingClassifier — are compared via 5-fold stratified cross-validation on an 80% training split only; the winner is chosen programmatically by PR-AUC rather than hardcoded.
- **3. Honest evaluation:** the winning pipeline is refit once on the full training split and scored exactly once on an untouched 20% holdout. The deployed model artifact is the literal object that produced the reported test metrics — it is never retrained on more data afterward, so the numbers in this report are true of what is actually served.
- **4. Explainability and serving:** a SHAP explainer (LinearExplainer or TreeExplainer, chosen automatically to match whichever model type won) is attached to the fitted pipeline. A FastAPI service exposes /predict, /health, and /model/info endpoints, validates every request, logs every prediction, and ships with a Dockerfile for containerized deployment.

- **5. Accessible demo interface:** the served page itself (the API's own root URL) is written for a general audience, not just engineers. It frames the system as an "AI Fraud Guard checking a purchase," explains the class-imbalance problem with a smoke-detector analogy, and phrases every model decision in plain language — "this raised suspicion a lot" rather than a raw SHAP value — while keeping exact technical values (feature names, contributions, the calibration threshold) available behind an opt-in "for technical reviewers" toggle for anyone auditing the underlying rigor. Visitors can also edit the transaction amount and see the model respond live, not just replay two fixed examples.

The API's root route (/) also serves a self-contained, click-through demo page — no separate frontend to run. It loads a real confirmed-fraud or confirmed-legit transaction from the held-out test set, sends it to /predict, and renders the live probability, verdict, and SHAP factors returned by the model. Nothing on that page is mocked or pre-computed. The four screenshots below are of the actual running application, not mockups.

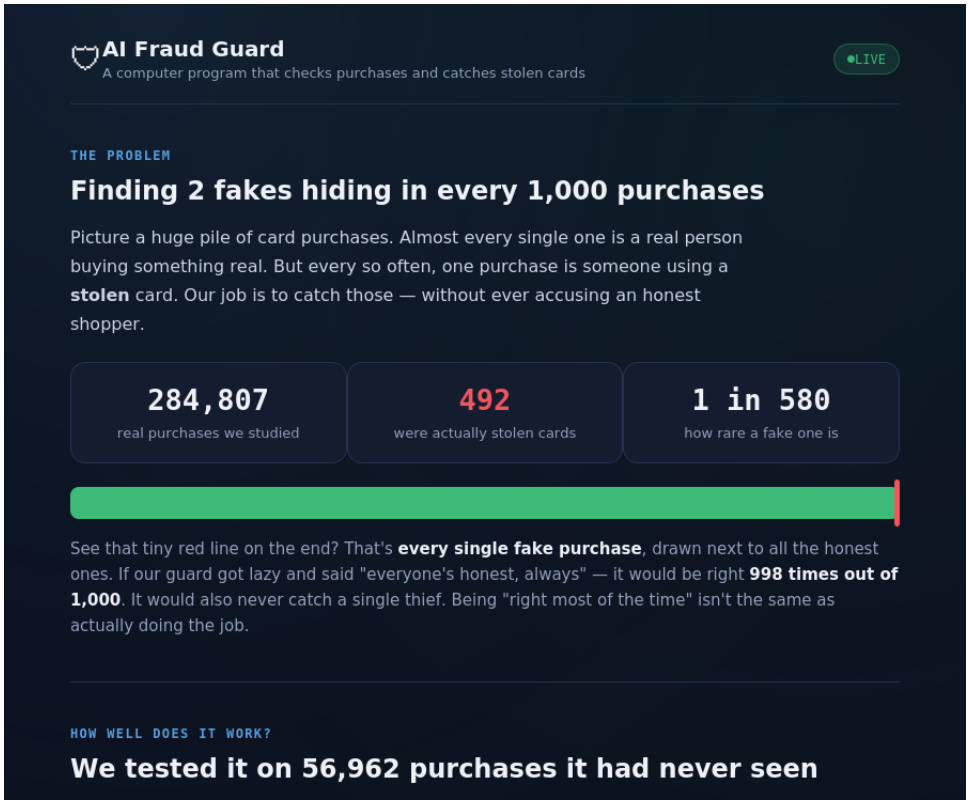


Figure 3 — The demo's opening framing: the class-imbalance problem stated in plain numbers, before any model output is shown.

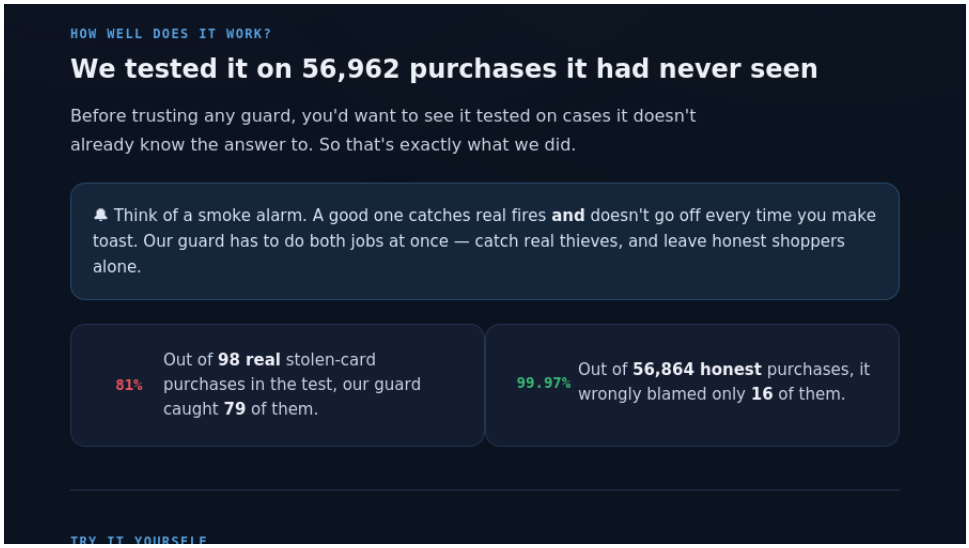


Figure 4 — Honest test-set results shown as a plain-language count ("caught 79 of 98") instead of a metric a general audience would need to decode.



Figure 5 — A real confirmed-fraud transaction scored live: flagged, with each of the 5 contributing signals named and ranked rather than shown as a bare feature code.



Figure 6 — The same flow on a real confirmed-legitimate transaction: cleared, with a visibly different mix of signals — proof the explanation changes with the input rather than being a canned message.

7. Key Insights Surfaced

Insight 1 — ROC-AUC looks solved; PR-AUC tells the truth

Both candidate models post a ROC-AUC around 0.97, which on its own would suggest the problem is nearly solved. PR-AUC tells a different story (0.71–0.76 depending on split and model — see Figures 7–8). The gap is mechanical, not a modeling error: ROC-AUC's false-positive-rate denominator is the entire legitimate-transaction population, so even a few dozen false alarms barely move it. Precision divides by flagged transactions instead, where the same false alarms are a much larger share of a much smaller number.

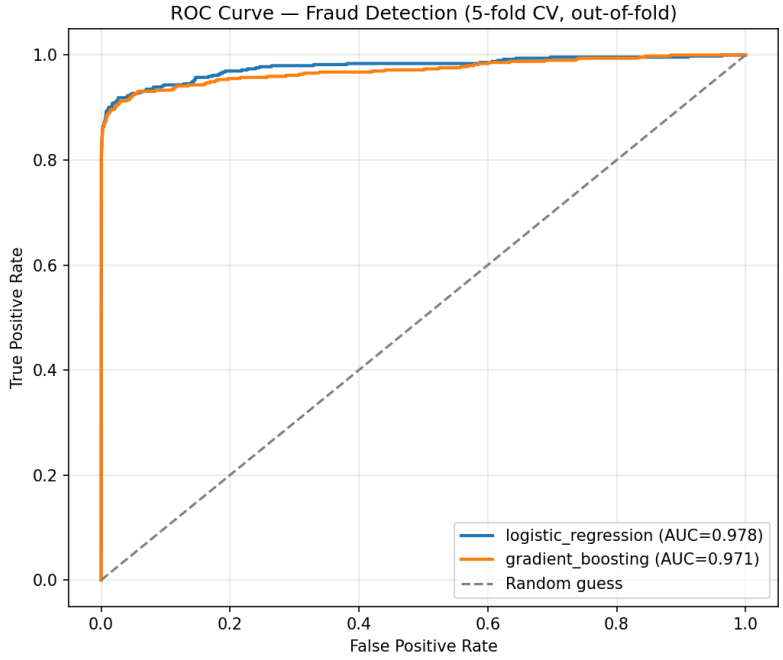


Figure 7 — ROC curves (full-dataset comparison): both models look close to solved (AUC ≈ 0.97–0.98).

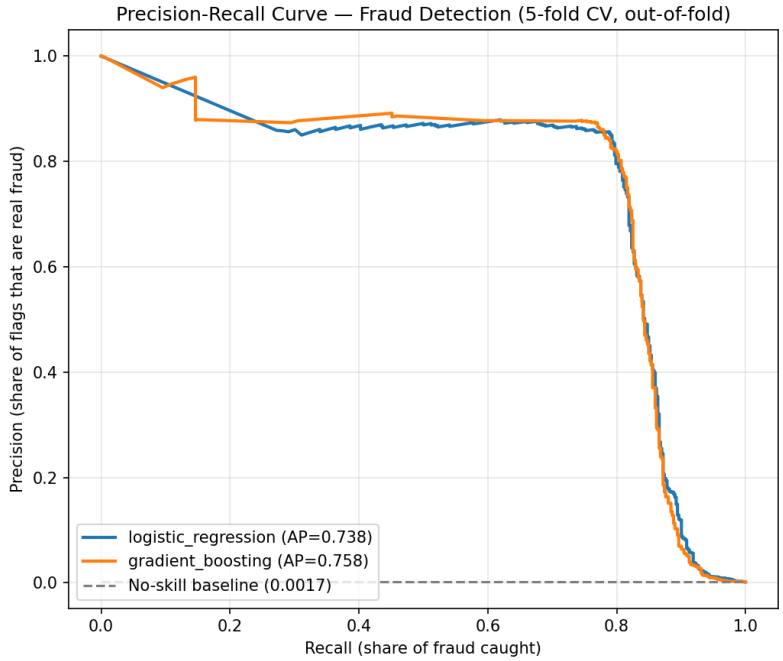


Figure 8 — The same two models' PR curves: a far more honest picture at 0.17% fraud prevalence (AP ≈ 0.74–0.76 on the full-dataset comparison shown here).

Insight 2 — class_weight="balanced" breaks the 0.5 threshold

The pipeline uses class_weight="balanced" to cope with the severe imbalance rather than resampling. Reweighting the loss as if classes were roughly 50/50 also skews predicted probabilities away from the dataset's true 0.173% prior. Using the conventional 0.5 probability cutoff on this pipeline's logistic regression model gives roughly 6% precision at 91% recall — close to unusable for an analyst. Reading the threshold off the out-of-fold precision-recall curve instead recovers 83.2% precision at a similar recall. This is a general trap: whenever class_weight="balanced" or resampling is used for an imbalanced problem, the default 0.5 threshold cannot be trusted without checking the PR curve first.

Insight 3 — Model choice is genuinely sensitive to the data split

With only 492 confirmed fraud rows in the whole dataset (394 in the 80% training split used for model selection), which model "wins" is sensitive to the specific split evaluated. An exploratory comparison on the full dataset favoured gradient boosting; the honest comparison on the training split alone — the one that actually decided what got deployed —

favoured logistic regression, and that choice held up on the untouched test set. This is why the pipeline selects a model programmatically from cross-validation rather than hardcoding a preference.

Insight 4 — SHAP confirms the model learned real structure

SHAP analysis identifies V4, V10, V14, V12, and V17 as the strongest overall drivers of the model's fraud predictions (Figure 9). These line up with patterns independently reported in public analyses of this same dataset — a reasonable sanity check that the model has learned genuine structure rather than noise. Because V1–V28 are anonymized PCA components, this ranking cannot be translated into a business reason the way it could on raw features; that mapping was deliberately destroyed by the dataset's creators to protect cardholder confidentiality.

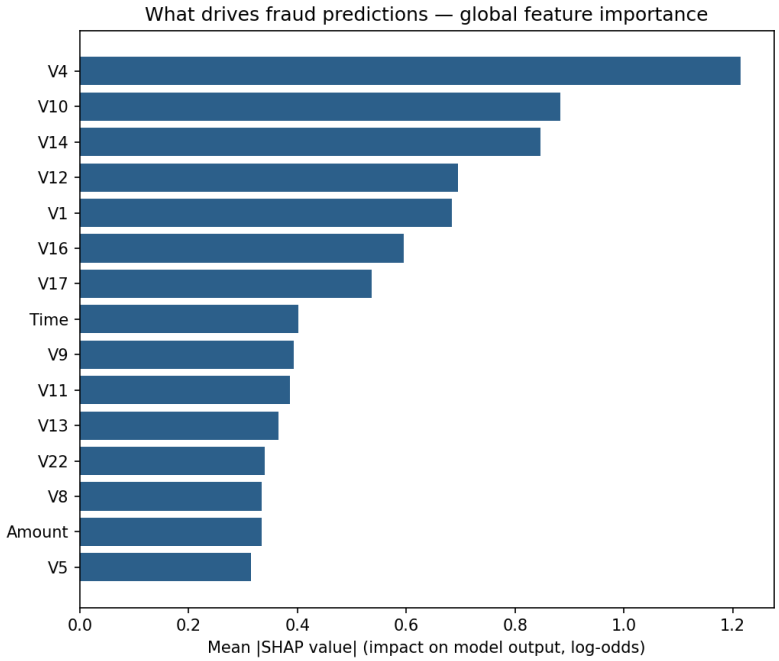


Figure 9 — Global SHAP feature importance: what drives fraud predictions overall.

Insight 5 — Outlier capping had to be selective

Capping outliers only on Amount, and explicitly not on V1–V28, turned out to matter. Fraudulent transactions in this dataset are disproportionately found in the tails of the PCA-component distributions; capping them would have clipped away the exact signal the model needs. Amount, by contrast, is raw and right-skewed (a maximum of $25,691 against a 75th percentile of $77), and capping it (IQR × 3) affected 6.65% of transactions without touching any anonymized feature.

8. Business Impact and Recommendations

At its tuned threshold, the deployed model catches 80.6% of confirmed fraud in the held-out test set while keeping 83.2% of its flags accurate: out of 98 real fraud cases, 79 are caught and 19 are missed, and 16 legitimate transactions are flagged incorrectly out of 56,864. Translated into a review workflow, roughly 5 out of every 6 transactions an analyst is asked to review under this model are actually fraud — a substantial improvement in review-queue quality over a rules-only baseline, which typically produces far more false alarms per fraud caught.

Recommendations:

- **Move to a cost-weighted threshold:** replace the F1-optimized threshold with one based on real dollar figures for a missed fraud versus a false decline once those figures are available — F1 treats the two errors as equally costly, which is rarely true in practice.

- **Treat threshold changes as a business decision:** the precision-recall curve is the right artifact for that conversation, since it shows the full trade-off, not just the currently deployed point on it.
- **Route the SHAP explanation to analysts, not just the score:** the top contributing factors give a fraud analyst a starting point instead of a bare probability.
- **Establish monitoring and a retraining cadence before production use:** all training data is from a single 48-hour window in 2013, and fraud patterns move.

9. Challenges and Solutions

Challenge	Solution
Severe class imbalance made accuracy meaningless (99.8% achievable by flagging nothing)	Adopted PR-AUC as the primary metric, class_weight="balanced" during training, and a threshold tuned from the out-of-fold precision-recall curve
Risk of preprocessing statistics leaking from test data into training	Wired imputation, capping, and scaling into a single Pipeline/ColumnTransformer, fit only inside CV folds or the training split — never on the full dataset
A 200-tree Random Forest took ~200s per CV fold on the available single-core environment	Swapped to HistGradientBoostingClassifier: comparable-or-better PR-AUC in ~5s per fold
class_weight="balanced" skewed predicted probabilities, silently breaking the 0.5 threshold	Selected the operating threshold explicitly from the PR curve and persisted it alongside the model
Explaining a model whose type (linear vs. tree-based) could change on a future retrain	Built a model-family-aware explainer that auto-selects LinearExplainer or TreeExplainer to match whatever is deployed

10. Outcome of Case Study

This case study delivered a complete, tested, and documented fraud detection system, not only a trained model:

- A leakage-safe Scikit-learn pipeline (custom outlier-capping transformer + ColumnTransformer) validated by 17 automated tests
- An honestly evaluated model — PR-AUC 0.708, ROC-AUC 0.971, 83.2% precision and 80.6% recall at its tuned threshold — measured on a 56,962-transaction holdout the model never influenced in any way
- A real-time explainability layer: every prediction ships with its top 5 contributing features, in original transaction units, via exact SHAP values
- A working REST API (FastAPI) with input validation, structured audit logging, and interactive documentation, packaged with a Dockerfile and docker-compose for containerized deployment
- A click-through live demo page served at the API's own root route, for in-person or remote demonstration without needing to write any curl commands
- A model card documenting intended use, explicit out-of-scope uses, evaluation methodology, and threshold governance

Live prototype: **AI Fraud Guard — Live Demo**
Deployed via Render's free Docker web-service tier (no code changes required beyond a $PORT environment-variable fix already applied) — see DEPLOYMENT.md in the project files for the exact, verified steps. If no live URL is filled in above, the same system is fully demonstrable locally: run uvicorn api:app and open the root URL in a browser for the identical demo experience.

11. Conclusion

The central lesson of this case study is that, on a severely imbalanced problem like fraud detection, evaluation methodology and leakage-safety determine whether a reported result means anything at all — they matter as much as which model is chosen. A pipeline that cannot show its preprocessing was fit only on training data, that reports accuracy instead of precision-recall, or that assumes a 0.5 threshold without checking a PR curve first can look excellent on paper and fail immediately in production.

Combining an honest held-out evaluation with per-prediction explainability turns a notebook experiment into something an analyst, a risk officer, or a regulator can actually trust and act on. Recommended next steps are a cost-weighted decision threshold, ongoing drift monitoring against live labels, and a periodic retraining cadence, since the fraud patterns of September 2013 are not guaranteed to be the fraud patterns of today.